

Name:	ID:	Section:
-------	-----	----------

Question 1 [C02] [10 Points]

An ancient civilization called **Xylos** protected its interstellar systems using DNA-based locks. Each lock accepts a DNA string **S**, which contains only the characters **A, T, C, G**, and always has an even length between 6 and 50. To open the lock, the string must be decoded correctly.

The decoding is done **two characters (A block) at a time**, from left to right.

- **First character of each block**
 1. Normally, apply the complement rule: $A \leftrightarrow T, C \leftrightarrow G$
 2. If the previous decoded character was G, apply rotation instead: $A \rightarrow G, C \rightarrow T, G \rightarrow A, T \rightarrow C$ (Trap rule)
- **Second character of each block**
 1. Normally, apply rotation: $A \rightarrow G, C \rightarrow T, G \rightarrow A, T \rightarrow C$
 2. If the first decoded character of the same block is A, apply the complement instead.
- Append the decoded characters in order to form the final string.

Print the final decoded DNA string.

Input	Output
ATCGAT	TCGATC
GATACA	CGCGTG

Test Case 2 Explanation:

Block 1 (G A):

First character G $\rightarrow$ Complement Rule $\rightarrow$ C (prev = none, normal complement)

Second character A $\rightarrow$ Rotation Rule $\rightarrow$ G

Result so far: CG

Block 2 (T A):

Previous decoded character = G $\rightarrow$ Trap Rule applies, first character uses Rotation instead of Complement $T \rightarrow C$

Second character A $\rightarrow$ Rotation Rule $\rightarrow$ G (first decoded $\neq$ A, normal rotation)

Result so far: CGCG

Block 3 (C A):

Previous decoded character = G $\rightarrow$ Trap Rule applies, first character uses Rotation $C \rightarrow T$

Second character A (first decoded = T $\rightarrow$ normal rotation) $\rightarrow$ G

Final decoded string: CGCGTG

Name:	ID:	Section:
-------	-----	----------

Question 1 [C02] [10 Points]

A lost civilization called **Zyron** secured their space stations using DNA-based locks. Each lock takes a DNA string **S**, which contains only the letters **A**, **T**, **C**, **G** and always has an even length between 6 and 50. To unlock a station, the string must be decoded correctly.

The decoding is done **two characters at a time** from left to right.

- **First character of each block:**
 1. Normally, apply the complement rule: $A \leftrightarrow T$, $C \leftrightarrow G$
 2. If the previous decoded character was **G**, apply rotation instead:
 $A \rightarrow G$, $C \rightarrow T$, $G \rightarrow A$, $T \rightarrow C$ (Trap Rule)
- **Second character of each block:**
 1. Normally, apply rotation: $A \rightarrow G$, $C \rightarrow T$, $G \rightarrow A$, $T \rightarrow C$
 2. If the first decoded character of the same block is **A**, apply complement instead (Mirror Rule)

Print the fully decoded DNA string

Input	Output
TACGGA	ATGACG

Explanation:

Block 1 (T A):

First character T $\rightarrow$ Complement Rule $\rightarrow$ A (prev = none, normal complement)

Second character A $\rightarrow$ Mirror Rule applies (first decoded = A) $\rightarrow$ T

Result so far: AT

Block 2 (C G):

Previous decoded character = T $\rightarrow$ Complement Rule $\rightarrow$ C $\rightarrow$ G

Second character G $\rightarrow$ Rotation Rule $\rightarrow$ A (first decoded $\neq$ A, normal rotation)

Result so far: ATGA

Block 3 (G A):

Previous decoded character = A $\rightarrow$ Complement Rule $\rightarrow$ G $\rightarrow$ C

Second character A $\rightarrow$ Rotation Rule $\rightarrow$ G (first decoded $\neq$ A, normal rotation)

Final decoded string: ATGACG

CSE110 Spring 2024 Lab Exam - 0X Tentative Solutions and Rubrics

In case of any circumstance not present in the rubrics, feel free to discuss in assigned slack channel. This rubric is provided to maintain a similar marking scheme throughout all sections.

[SET-A]

SOLUTION

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();

        String result = "";
        char prevDecoded = ' ';
        int i = 0;

        while (i < s.length()) {

            // First character
            char first = s.charAt(i);
            char decodedFirst;

            if (prevDecoded == 'G') { // rotation
                if (first == 'A') decodedFirst = 'G';
                else if (first == 'C') decodedFirst = 'T';
                else if (first == 'G') decodedFirst = 'A';
                else decodedFirst = 'C'; // T
            } else { // complement
                if (first == 'A') decodedFirst = 'T';
                else if (first == 'T') decodedFirst = 'A';
                else if (first == 'C') decodedFirst = 'G';
                else decodedFirst = 'C'; // G
            }

            result = result + decodedFirst;

            // Second character
            char second = s.charAt(i + 1);
            char decodedSecond;

            if (decodedFirst == 'A') { // complement
                if (second == 'A') decodedSecond = 'T';
                else if (second == 'T') decodedSecond = 'A';
                else if (second == 'C') decodedSecond = 'G';
                else decodedSecond = 'C'; // G
            } else { // rotation
                if (second == 'A') decodedSecond = 'G';
                else if (second == 'C') decodedSecond = 'T';
                else if (second == 'G') decodedSecond = 'A';
                else decodedSecond = 'C'; // T
            }

            result = result + decodedSecond;
            prevDecoded = decodedSecond;

            i = i + 2;
        }
        System.out.println(result);
    }
}
```

Rubric For Set A and Set B

SL	Points To Meet	Marks [10]
1	Correctly providing input and output and other things	2
2	Correctly implementing the complement rule for the first char	2
3	Correctly implementing the trap rule for the first char	2
4	Correctly implementing the rotation rule for the second char	2
6	Correctly implementing the complement rule for the second char	2

[SET-B]

SOLUTION

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        String result = "";
        char prevDecoded = ' ';
        int i = 0;

        while (i < s.length()) {

            // First character
            char first = s.charAt(i);
            char decodedFirst;

            if (prevDecoded == 'G') { // Trap rule: rotation instead of complement
                if (first == 'A') decodedFirst = 'G';
                else if (first == 'C') decodedFirst = 'T';
                else if (first == 'G') decodedFirst = 'A';
                else decodedFirst = 'C'; // T
            } else { // Normal complement
                if (first == 'A') decodedFirst = 'T';
                else if (first == 'T') decodedFirst = 'A';
                else if (first == 'C') decodedFirst = 'G';
                else decodedFirst = 'C'; // G
            }

            result = result + decodedFirst;

            // Second character
            char second = s.charAt(i + 1);
            char decodedSecond;

            if (decodedFirst == 'A') { // Mirror rule: complement instead of rotation
                if (second == 'A') decodedSecond = 'T';
                else if (second == 'T') decodedSecond = 'A';
                else if (second == 'C') decodedSecond = 'G';
                else decodedSecond = 'C'; // G
            } else { // Normal rotation
                if (second == 'A') decodedSecond = 'G';
                else if (second == 'C') decodedSecond = 'T';
                else if (second == 'G') decodedSecond = 'A';
                else decodedSecond = 'C'; // T
            }

            result = result + decodedSecond;
            prevDecoded = decodedSecond;

            i = i + 2;
        }

        System.out.println(result);
    }
}
```

